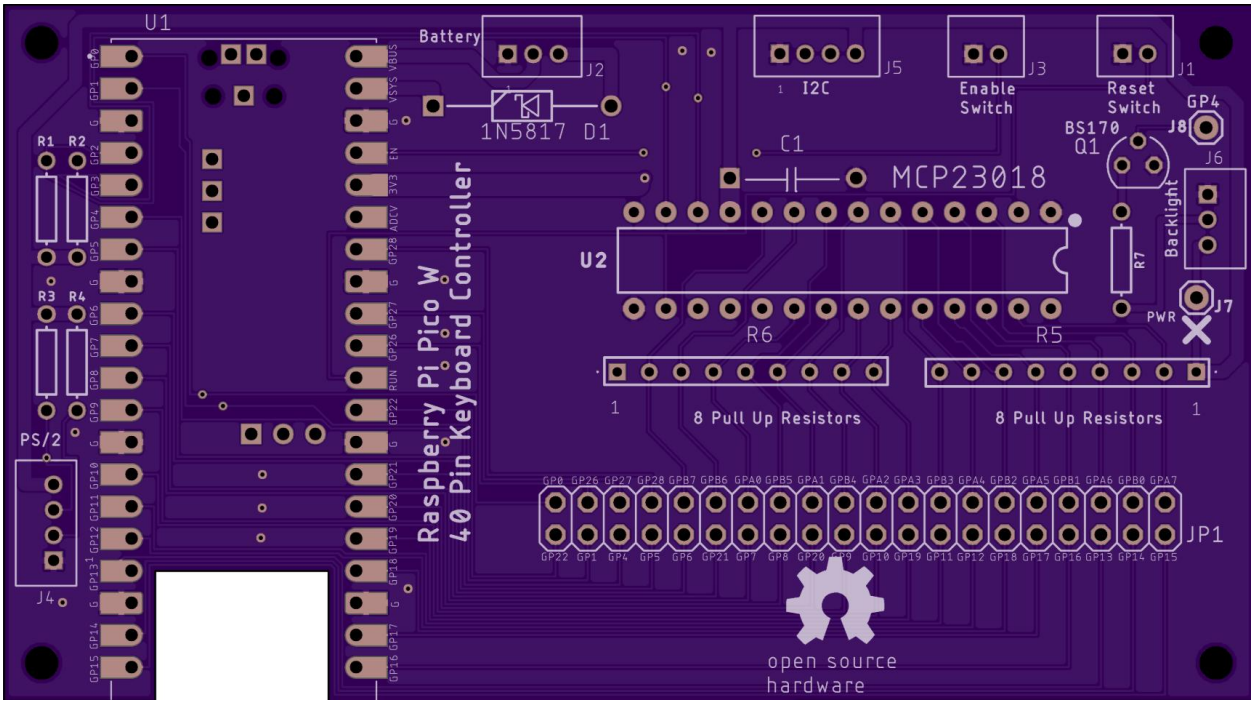
Raspberry Pi Pico W with Port Expander

I have designed a laptop keyboard controller board using a Raspberry Pi Pico W that includes an MCP23018 port expander. This increases the 26 pin GPIO limit of a stand-alone Pi Pico to 40 GPIO's (16 from the port expander and 24 from the Pico). All components for this board are through-hole for easy soldering including the FPC adapter board. The Eagle board and schematic files along with a zipped Gerber layout file are at my repo.

The OSHPark rendition of the Raspberry Pi Pico W port expander board is shown below:

The size is 99mm x 55mm. JLCPCB.com cost is $6 for 5 boards (with slow shipping).



The cutout in the board gives better reception for the Bluetooth antenna on the Pico. The Pico can be mounted with header pins or soldered directly to the board for a lower profile.

The two rows of 20 pins at JP1 are for mounting an FPC adapter board. Choose a board with the correct pitch and pin count that matches your keyboard's FPC cable. Don't get a board with pre-soldered header pins or you'll have to mount it underneath the Pico board. Solder your own header pins with the adapter board sitting on top. If your cable has fewer than 40 pins, install the correct size FPC adapter board in the pads on the right side, leaving the pads on the left side empty. This will free up Pico GPIO pins for other uses such as touchpad and backlight control.

Bluetooth Low Energy (BLE) operation implies the controller will be running from a separate lithium battery with a charging circuit. USB 5 volts on the Pico VBUS pin is routed to pin 2 of a 3 pin JST connector at J2. This should be cabled (along with ground) to an off-board battery charger circuit. The nominal 3.7 volt battery output should be cabled to J2 pin 3. This feeds the anode of a 1N5817 Schottky diode at D1 that "or-ties" the battery voltage to the Pico's VSYS pin. When the USB cable is attached, a Schottky diode inside the Pico brings USB power to VSYS. If the keyboard will only be used for USB, it will always have power so Schottky diode D1 and the J2 connector are not needed.

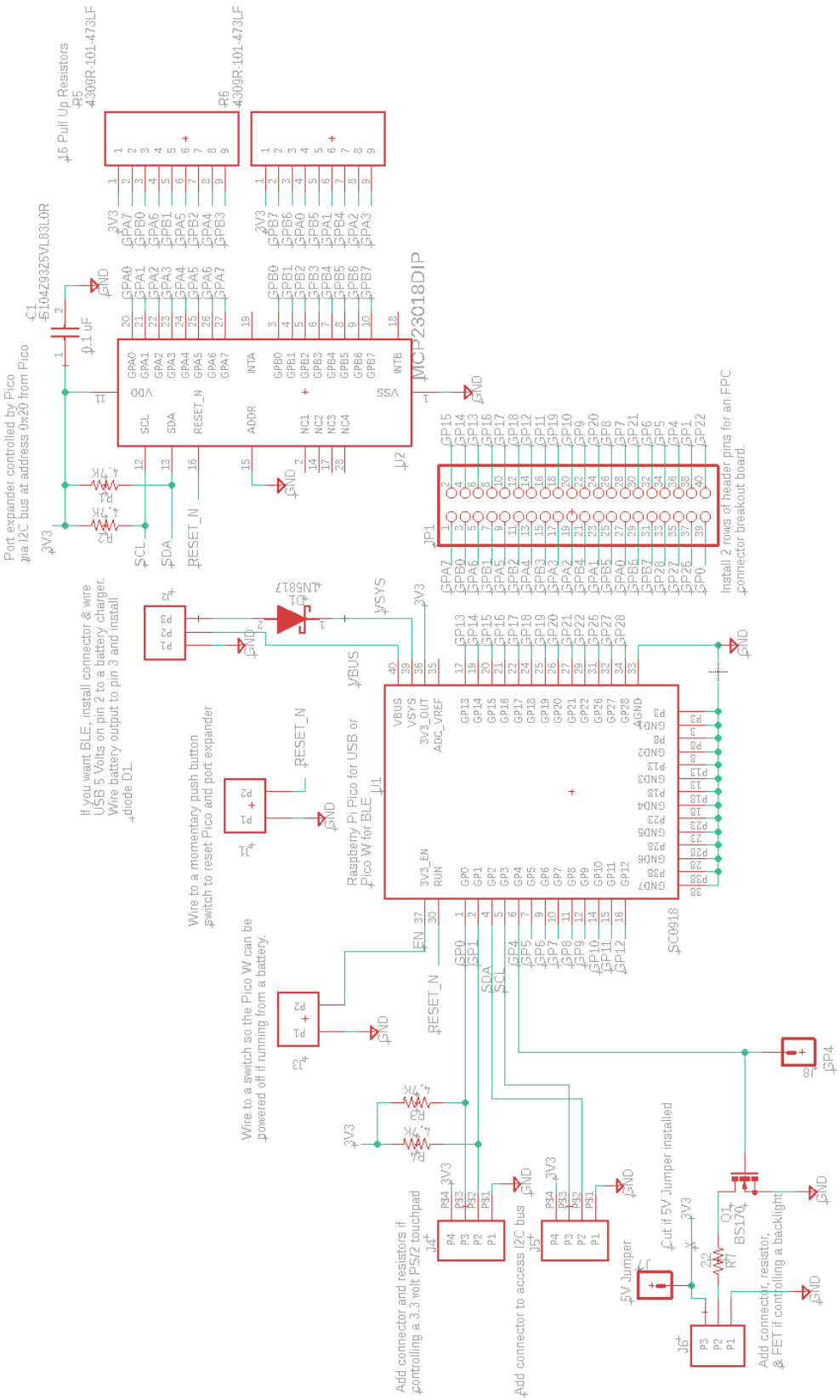
The EN signal at Pico Pin 37 is routed to a 2 pin JST connector at J3. If you want to power down the Pico to save the battery, add a switch to connect the EN signal to ground.

Resistors R1 and R2 are the I2C clock and data pullups and should be 4.7KΩ for running the bus at 400KHz. The I2C bus controls the MCP23018 port expander and it is also routed to the 4 pin JST connector at J5. This connector is only needed if interfacing to an I2C touchpad.

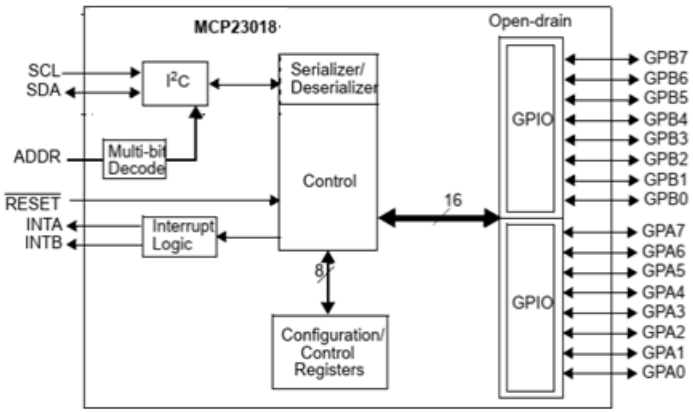
Clock and data pullup resistors R3 and R4 are 4.7KΩ and are driven by GPIO 0 and 1 of the Pico. These resistors and the 4 pin JST connector at J4 are only needed if interfacing to a PS/2 touchpad. Note that using these GPIO's for PS/2 reduces the available keyboard FPC pins to 38 instead of 40.

An additional keyboard pin can be repurposed for controlling a backlight LED by installing a 3 pin JST connector at J6. This connector provides 3.3 volts, ground, and an open drain N Channel FET with a current limit resistor for controlling an LED. Many laptop keyboards provide the backlight LED anode and cathode on multiple pins of a small FPC cable. Use a small FPC adapter board to wire the anode pins to 3.3 volts at J6 pin 3 and the cathode pins to J6 pin 2. Install a current limit resistor at R7 and a BS170 NFET at Q1. Size the resistor so the current at max brightness stays well under the Pico's 300ma limit. Pico GPIO 4 can be programmed to output a PWM signal to the gate of the NFET which turns on and off the high current to the backlight LED.

The Eagle Schematic is given below:



The MCP23018 port expander diagram is shown below.



My board powers this chip from the 3.3 volt regulator at Pico pin 36. The SDA and SCL signals are driven from the Pico I2C pins at GP2 and GP3. The 4.7KΩ pullup resistors (and short traces) allow the I2C bus to operate at 400KHz. The Address pin is grounded to set the I2C address for the port expander at 0x20. The MCP23018 RESET_n pin is tied to the Pico's RUN pin and to a 2 pin JST connector at J1 that can be wired to a push button reset switch. The INTA and INTB pins are not used. Each GP pin of the port expander can be programmed over I2C as an input or an output. Outputs are open drain which can only drive low or float. Inputs can be programmed to have a 100KΩ pull up resistor which would result in a very slow rise time when a key is released. To improve the rise time, all 16 port expander pins are wired to 47KΩ pullup resistors. The 100KΩ and 47KΩ parallel resistors result in 32KΩ pull ups on the port expander inputs. The Pico can program its GPIO inputs to have pull ups and their nominal value is 50KΩ. I've done a lot of testing with Pico controlled keyboards and the 50KΩ pullup value has not caused any problems. QMK defaults to waiting 30 microseconds before reading an input pin. If more time is needed due to a slow rise time, you can specify a longer delay with this command added to the top section of the keyboard.json file:

"io_delay": <delay number in microseconds>

Programming – This board can be programmed for USB or BLE operation using Arduino C++ code and the Earle Philhower library. QMK can also be used because it has modules for controlling the MCP23018 Port Expander. Unfortunately, this board will not work with KMK because it does not support port expanders.